Variational Autoencoders vs. Denoising Diffusion Probabilistic Models

Implementation from Scratch, Benchmark and Analysis on CIFAR-10

Eyad Elnakib

Code, results and logs: https://github.com/Eyad-Elnakib1/Generative-models-VAE-DDPM

October 2026

Abstract

We implement two foundational deep generative models from scratch in PyTorch: the Variational Autoencoder (VAE) [1] and the Denoising Diffusion Probabilistic Model (DDPM) [2]. We train three models on CIFAR-10: the DDPM in its original configuration, a VAE with the paper's own one-layer MLP, and a VAE with a convolutional network whose capacity matches the DDPM's U-Net. All three are evaluated under one protocol covering sample quality and diversity (FID, IS, KID, precision/recall), likelihood (bits/dim), sampling cost, thematic consistency, memorisation and a blind rating.

The DDPM reaches an FID of 6.58, an Inception Score of 8.59 and a test likelihood bound of 3.79 bits/dim. The bound is within 0.04 bits/dim of the published value, after 19% of the original training. The convolutional VAE reaches an FID of 121.9, an IS of 3.88 and 5.83 bits/dim. The VAE generates images about 1,700× faster, because it needs one network evaluation per image instead of 1000. With 10-step DDIM sampling the DDPM is still 6.6× better in FID than the VAE.

We trace every observed difference to one design choice. The VAE *learns* an amortised encoder into a low-dimensional latent space and decodes in a single factorised step. The DDPM *fixes* its encoder to gradual Gaussian noising and decodes through a chain of 1000 small, learned denoising steps.

Contents

1 Introduction **3**

2 Theoretical Background **4**

2.1 Latent-variable models and the evidence lower bound 4
2.2 Variational Autoencoder 4
2.3 Denoising Diffusion Probabilistic Model 5
2.4 A unifying view 6

3 Model Architectures **7**

3.1 Shared building blocks 7
3.2 Model A — VAE-MLP: the original network of Kingma & Welling 8
3.3 Model B — VAE-Conv: the same method with a convolutional network 9
3.4 Model C — DDPM: the noise-prediction U-Net of Ho et al. 10
3.5 The three architectures side by side 12

4 Training Procedure **12**
4.1 Algorithms . . . 12
4.2 Hyper-parameters and compute . . . 13

5 Evaluation Protocol **14**
5.1 Metrics . . . 14
5.2 Protocol and fairness . . . 15

6 Quantitative Results **15**
6.1 Main comparison . . . 15
6.2 Speed–quality trade-off . . . 16
6.3 Quality versus diversity . . . 17
6.4 Training dynamics and the VAE's prior hole . . . 18
6.5 Likelihood decomposition . . . 19
6.6 Ablations without extra training, and memorisation . . . 20

7 Qualitative Analysis **20**
7.1 Realism . . . 21
7.2 Diversity and thematic consistency . . . 22
7.3 How each model generates . . . 23
7.4 Latent spaces . . . 24
7.5 Blind rating by a vision-language model . . . 24

8 Discussion: the Main Differences between the Three Models **25**
8.1 VAE-MLP vs. VAE-Conv: the effect of the network . . . 25
8.2 VAE-Conv vs. DDPM: the effect of the generative approach . . . 26
8.3 Summary . . . 26

9 Limitations and Threats to Validity **27**

10 Conclusion **28**

A Reproducibility **30**

1 Introduction

Deep generative models learn a distribution $p_\theta(\mathbf{x})$ from data and can draw new samples from it. Two families laid the foundations of modern image generation:

- **Variational Autoencoders** (VAEs) [1] fit a latent-variable model by maximising a variational lower bound with an amortised inference network.
- **Denoising Diffusion Probabilistic Models** (DDPMs) [2, 3] learn to invert a fixed, gradual noising process.

Both are trained by maximising an evidence lower bound (ELBO), yet they behave very differently in practice.

This report implements both models from first principles, trains them on CIFAR-10 [4], and compares them quantitatively and qualitatively, with the goal of explaining the *main difference* between the two approaches.

Contributions.

1. **Faithful implementations from scratch** (Sections 3 and 4). The DDPM reproduces the paper's CIFAR-10 configuration exactly (35.7M-parameter U-Net); only the training length is shortened. The VAE follows the original method exactly and is instantiated with two networks: the paper's MLP and a convolutional network built from the U-Net's blocks.
2. **A shared, validated evaluation protocol** (Section 5). It covers FID, IS, KID, precision/recall, likelihood, the speed–quality trade-off, classifier-based thematic consistency, memorisation and a blind rating. Before any model was evaluated, the metrics were checked to reproduce published reference values on real data.
3. **A mechanistic comparison** (Sections 6 to 8). It links each empirical difference (blur, diversity, the "prior hole", likelihood, sampling cost) to the structure of the two models.

Dataset. We use CIFAR-10 [4]: 50,000 training and 10,000 test images of 32×32 RGB pixels in 10 classes. It is the DDPM paper's main benchmark, so published numbers exist to check our implementation against. Its classes make thematic consistency measurable, and the Inception Score is only meaningful on ImageNet-like natural images such as these. Every training run and every speed measurement used one NVIDIA A100-SXM4-80GB (Google Colab), so compute and timings are directly comparable.

Models compared. Throughout the report we compare three models:

- **VAE-MLP**: the VAE with the network of Kingma and Welling [1].
- **VAE-Conv**: the same VAE method with a convolutional network of comparable size to the DDPM. This is the fair counterpart to the DDPM.
- **DDPM**: the model of Ho et al. [2].

Code and data availability. All code, configurations, training logs, raw evaluation outputs and figures are public at `https://github.com/Eyad-Elnakib1/Generative-models-VAE-DDPM`. Every number in this report is produced by the scripts in that repository.

2 Theoretical Background

2.1 Latent-variable models and the evidence lower bound

A latent-variable model introduces unobserved variables $\mathbf{z}$ and defines

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}. \tag{1}$$

Maximum likelihood requires $\log p_\theta(\mathbf{x})$, which is intractable for neural decoders. For any distribution $q(\mathbf{z} \mid \mathbf{x})$, Jensen's inequality gives the *evidence lower bound*

$$\log p_\theta(\mathbf{x}) \;\geq\; \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z})\big] - D_{\mathrm{KL}}(q(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})) \;=\; \mathcal{L}_{\mathrm{ELBO}}(\mathbf{x}), \tag{2}$$

with equality when $q(\mathbf{z} \mid \mathbf{x})$ is the true posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$. Both models in this report maximise a bound of this form. They differ in *what* the latent variables are and *whether* q is learned.

2.2 Variational Autoencoder

Generative model. A latent code $\mathbf{z} \in \mathbb{R}^{d_z}$ is drawn from a standard normal prior and decoded by a neural network into a diagonal Gaussian over pixels:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}), \qquad p_\theta(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}\big(\mathbf{x};\, \boldsymbol{\mu}_\theta(\mathbf{z}),\, \mathrm{diag}\, \boldsymbol{\sigma}^2_\theta(\mathbf{z})\big). \tag{3}$$

Inference model (encoder). The intractable posterior is approximated by an amortised Gaussian

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\mathbf{z};\, \boldsymbol{\mu}_\phi(\mathbf{x}),\, \mathrm{diag}\, \boldsymbol{\sigma}^2_\phi(\mathbf{x})\big), \tag{4}$$

whose parameters are predicted by an encoder network in a single forward pass.

Reparameterisation. To back-propagate through the sampling of $\mathbf{z}$, the sample is written as a deterministic function of ϕ and an independent noise variable:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{5}$$

The expectation in Equation (2) is estimated with a single sample ($L = 1$), as in the original paper.

The two ELBO terms in closed form. With $D = 3 \times 32 \times 32 = 3072$ pixel dimensions and d_z latent dimensions, the reconstruction term of one sample is

$$-\log p_\theta(\mathbf{x} \mid \mathbf{z}) = \frac{1}{2} \sum_{i=1}^{D} \left[\log 2\pi + \log \sigma^2_{\theta,i}(\mathbf{z}) + \frac{\big(x_i - \mu_{\theta,i}(\mathbf{z})\big)^2}{\sigma^2_{\theta,i}(\mathbf{z})} \right], \tag{6}$$

and the KL divergence between two diagonal Gaussians is analytic:

$$D_{\mathrm{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})) = \frac{1}{2} \sum_{j=1}^{d_z} \left(\mu^2_{\phi,j} + \sigma^2_{\phi,j} - \log \sigma^2_{\phi,j} - 1 \right). \tag{7}$$

Both sums run over *all* dimensions before averaging over the batch, so the KL weight is exactly $\beta = 1$. A common implementation error averages Equation (6) per pixel but sums Equation (7). That silently multiplies the KL weight by $D = 3072$ and causes posterior collapse; a unit test in the repository guards against it.

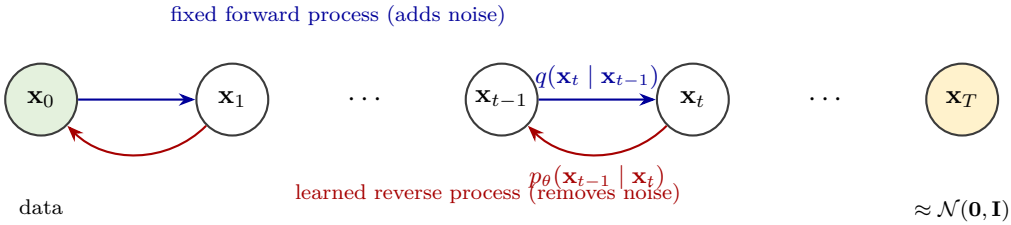


Figure 1: The DDPM as a Markov chain. The forward process q is fixed and gradually destroys the image; a single network, shared across all t, parameterises the reverse process p_θ, which generates images from noise.

Generation. A new image is obtained by drawing $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and outputting $\boldsymbol{\mu}_\theta(\mathbf{z})$. This takes *one* decoder evaluation.

Discrete data and bits/dim. Pixels are integers $k \in \{0, \dots, 255\}$. To make a continuous density a valid bound on the discrete likelihood, we use uniform dequantisation:

$$x = \frac{k + u}{128} - 1, \qquad u \sim \mathcal{U}[0, 1), \tag{8}$$

which maps every bin to an interval of width $w = 2/256$ in $[-1, 1)$. Since $P(k) = \int_{\text{bin}} p(x)\, dx \geq w \exp \mathbb{E}_u[\log p(x)]$, the negative ELBO $\mathcal{B}$ (in nats) of an image converts into a bound in bits per dimension:

$$\text{bpd} = \frac{\mathcal{B} + D \ln 128}{D \ln 2}. \tag{9}$$

2.3 Denoising Diffusion Probabilistic Model

Forward (diffusion) process. A fixed Markov chain adds Gaussian noise over $T = 1000$ steps (Figure 1):

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{1 - \beta_t}\, \mathbf{x}_{t-1},\ \beta_t \mathbf{I}\big), \qquad \beta_t = 10^{-4} + \frac{t - 1}{T - 1}\,(0.02 - 10^{-4}). \tag{10}$$

With $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, the marginal at any step is available in closed form, so $\mathbf{x}_t$ can be sampled in one shot during training:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{\bar{\alpha}_t}\, \mathbf{x}_0,\ (1 - \bar{\alpha}_t)\mathbf{I}\big) \iff \mathbf{x}_t = \sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{11}$$

With this schedule, $\bar{\alpha}_T \approx 4 \times 10^{-5}$, so $\mathbf{x}_T$ is almost exactly standard normal.

Forward-process posterior. Conditioned on $\mathbf{x}_0$, the reverse of a forward step is a tractable Gaussian:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_{t-1};\ \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0),\ \tilde{\beta}_t \mathbf{I}\big), \tag{12}$$

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\, \beta_t}{1 - \bar{\alpha}_t}\, \mathbf{x}_0 + \frac{\sqrt{\alpha_t}\,(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\, \mathbf{x}_t, \qquad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\, \beta_t. \tag{13}$$

Reverse (generative) process. A neural network parameterises Gaussian transitions that start from pure noise:

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1};\ \boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\ \sigma_t^2 \mathbf{I}\big), \tag{14}$$

with fixed variances $\sigma_t^2 = \beta_t$ ("fixed-large", used for CIFAR-10) or $\sigma_t^2 = \tilde{\beta}_t$ ("fixed-small").

Variational bound. The negative ELBO of the chain decomposes into a sum of per-step terms:

$$-\log p_\theta(\mathbf{x}_0) \;\le\; \underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_T \mid \mathbf{x}_0) \,\|\, p(\mathbf{x}_T))}_{L_T} \;+\; \sum_{t=2}^{T} \underbrace{\mathbb{E}_q \, D_{\mathrm{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \,\|\, p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t))}_{L_{t-1}} \\ \underbrace{-\,\mathbb{E}_q \log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)}_{L_0}. \tag{15}$$

L_T has no parameters, and each L_{t-1} is a KL divergence between two Gaussians. The final term L_0 uses a discretised Gaussian decoder that integrates the density over each pixel's bin (width $2/255$, with the outermost bins extended to $\pm\infty$), so the bound is a proper bound on the discrete likelihood.

Noise parameterisation and L_{simple}. Substituting Equation (11) into Equation (13) suggests predicting the noise instead of the mean:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \, \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right). \tag{16}$$

With this choice, L_{t-1} becomes a weighted squared error on the noise. Ho et al. [2] drop the weights and train with

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}\{1..T\},\, \mathbf{x}_0,\, \boldsymbol{\epsilon}} \Big[\big\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},\; t\big) \big\|^2 \Big], \tag{17}$$

a reweighted ELBO that emphasises the harder, noisier steps and gives the best samples. In our implementation the mean is computed through the predicted clean image $\hat{\mathbf{x}}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_\theta)/\sqrt{\bar{\alpha}_t}$, clipped to $[-1, 1]$, followed by $\boldsymbol{\mu}_\theta = \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \hat{\mathbf{x}}_0)$. Without clipping this is algebraically identical to Equation (16).

Ancestral sampling. Generation runs the reverse chain from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: $\mathbf{x}_{t-1} = \boldsymbol{\mu}_\theta(\mathbf{x}_t, t) + \sigma_t \mathbf{z}$ with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $t > 1$. This takes $T = 1000$ network evaluations per image.

DDIM sampling. Song et al. [5] show that the same trained network $\boldsymbol{\epsilon}_\theta$ also defines a deterministic, non-Markovian sampler on a subsequence $\tau_1 < \cdots < \tau_S$ of timesteps. For $\eta = 0$:

$$\mathbf{x}_{\tau_{i-1}} = \sqrt{\bar{\alpha}_{\tau_{i-1}}}\;\hat{\mathbf{x}}_0(\mathbf{x}_{\tau_i}) + \sqrt{1-\bar{\alpha}_{\tau_{i-1}}}\;\boldsymbol{\epsilon}_\theta(\mathbf{x}_{\tau_i}, \tau_i). \tag{18}$$

This trades quality for speed with *no retraining*, which we use to measure the speed–quality frontier.

2.4 A unifying view

Equation (15) is the ELBO of a *hierarchical VAE* with latents $\mathbf{x}_1, \ldots, \mathbf{x}_T$. Read this way, a DDPM is a VAE with three special properties:

1. the "encoder" $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)$ is fixed and has no parameters;
2. every latent has the same dimension as the image, so nothing is compressed;
3. the prior is matched by construction ($L_T \approx 0$), so the KL to the prior costs nothing.

The VAE makes the opposite choices: it has a single latent layer, a *learned* encoder, a compressed latent ($d_z \ll D$), and a KL term the encoder must actively pay for. Section 8 shows that these choices explain every difference we measure.

3 Model Architectures

All three models are built from scratch (`src/genmodels/`). The convolutional VAE and the DDPM share one library of building blocks, which follows the official DDPM implementation [2]. Using the same blocks keeps the comparison about the *generative approach* rather than about network design. Images enter every model as $3 \times 32 \times 32$ tensors scaled to $[-1, 1]$.

3.1 Shared building blocks

Group normalisation and Swish. Every block normalises its input with GroupNorm [6] (32 groups, $\epsilon = 10^{-6}$) and uses the Swish/SiLU activation:

$$\mathrm{GN}(\mathbf{h})_c = \gamma_c \frac{\mathbf{h}_c - \mu_{g(c)}}{\sqrt{\sigma^2_{g(c)} + \epsilon}} + b_c, \qquad \mathrm{SiLU}(a) = a \cdot \frac{1}{1 + e^{-a}}, \tag{19}$$

where μ_g and σ^2_g are computed per image over the channels of group g and all spatial positions. Unlike batch normalisation, this does not depend on the batch, which matters because the same network serves every noise level t.

Residual block. The Wide-ResNet [7, 8] block maps $\mathbf{h} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ to C_{out} channels at the same resolution:

$$\mathbf{a} = \mathrm{Conv}_{3\times3}\big(\mathrm{SiLU}(\mathrm{GN}(\mathbf{h}))\big) \big[+\ \mathbf{W}_t\ \mathrm{SiLU}(\mathbf{e}_t)\big], \tag{20}$$

$$\mathrm{Res}(\mathbf{h}) = \mathrm{Skip}(\mathbf{h}) + \mathrm{Conv}_{3\times3}\big(\mathrm{Dropout}(\mathrm{SiLU}(\mathrm{GN}(\mathbf{a})))\big). \tag{21}$$

- The bracketed term adds a projection of the timestep embedding $\mathbf{e}_t$, broadcast over $H \times W$. Only the DDPM uses it; the VAE's blocks omit it.
- Skip is the identity when $C_{\text{in}} = C_{\text{out}}$ and a 1×1 convolution otherwise.
- The second convolution is initialised near zero, so every block starts as the identity map. This stabilises training of deep stacks.

Self-attention. At 16×16 resolution (and in the bottleneck), a single-head spatial self-attention block [9] lets every position attend to all HW others. This captures global structure that 3×3 convolutions see only after many layers:

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{W}_{q,k,v}\ \mathrm{GN}(\mathbf{h}) \in \mathbb{R}^{HW \times C}, \qquad \mathrm{Attn}(\mathbf{h}) = \mathbf{h} + \mathbf{W}_o\ \mathrm{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{C}}\Big)\mathbf{V}, \tag{22}$$

where the $\mathbf{W}$ are 1×1 convolutions and $\mathbf{W}_o$ is initialised near zero.

Resampling. Downsampling is a stride-2 3×3 convolution, with TensorFlow-style asymmetric padding as in the official code. Upsampling is nearest-neighbour interpolation by $2\times$ followed by a 3×3 convolution.

Timestep embedding (DDPM only). The integer t is encoded with the Transformer sinusoidal embedding [9] of dimension $d = 128$ and passed through a two-layer MLP:

$$[\mathbf{s}_t]_i = \sin\big(t\,\omega_i\big), \qquad [\mathbf{s}_t]_{i+64} = \cos\big(t\,\omega_i\big), \qquad \omega_i = 10000^{-i/63}, \quad i = 0, \ldots, 63, \tag{23}$$

$$\mathbf{e}_t = \mathbf{W}_2\ \mathrm{SiLU}(\mathbf{W}_1\mathbf{s}_t) \in \mathbb{R}^{512}. \tag{24}$$

Initialisation. All convolutions and linear layers use Xavier-uniform initialisation (TensorFlow's `variance_scaling(fan_avg)`), with near-zero scale (10^{-10}) for the last layer of every residual branch and of every output head, as in the reference implementation.

3.2 Model A — VAE-MLP: the original network of Kingma & Welling

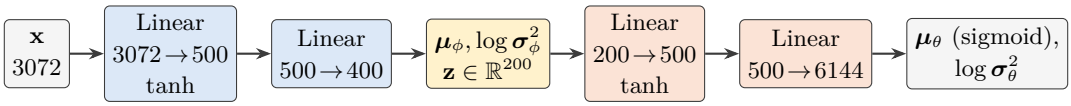


Figure 2: VAE-MLP: one tanh hidden layer of 500 units on each side, as in Kingma and Welling [1].

The image is flattened into a vector of $D = 3072$ values, so the network has no notion of spatial neighbourhood. Following the paper's Gaussian MLP (Appendix C.2):

$$\text{encoder:} \quad \mathbf{h} = \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \qquad [\boldsymbol{\mu}_\phi, \log \boldsymbol{\sigma}^2_\phi] = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2, \tag{25}$$

$$\text{decoder:} \quad \mathbf{h}' = \tanh(\mathbf{W}_3\mathbf{z} + \mathbf{b}_3), \qquad [\mathbf{m}, \mathbf{r}] = \mathbf{W}_4\mathbf{h}' + \mathbf{b}_4, \qquad \boldsymbol{\mu}_\theta = 2\,\mathrm{sigmoid}(\mathbf{m}) - 1. \tag{26}$$

The sigmoid constrains the decoder means, as in the paper; the factor $2(\cdot) - 1$ maps it to our $[-1, 1]$ pixel range. The decoder log-variance $\mathbf{r}$ is used through the soft lower bound described in Section 3.3. All parameters are initialised from $\mathcal{N}(0, 0.01^2)$.

Table 1: VAE-MLP layers. Total: **4,915,544** parameters.

Part	Layer	Output	Parameters
Encoder	Flatten	3072	0
	Linear $3072 \to 500$, tanh	500	1,536,500
	Linear $500 \to 2 \times 200$ ($\boldsymbol{\mu}_\phi$, $\log \boldsymbol{\sigma}^2_\phi$)	2×200	200,400
Decoder	Linear $200 \to 500$, tanh	500	100,500
	Linear $500 \to 2 \times 3072$ ($\boldsymbol{\mu}_\theta$, $\log \boldsymbol{\sigma}^2_\theta$)	$2 \times 3 \times 32 \times 32$	3,078,144

3.3 Model B — VAE-Conv: the same method with a convolutional network

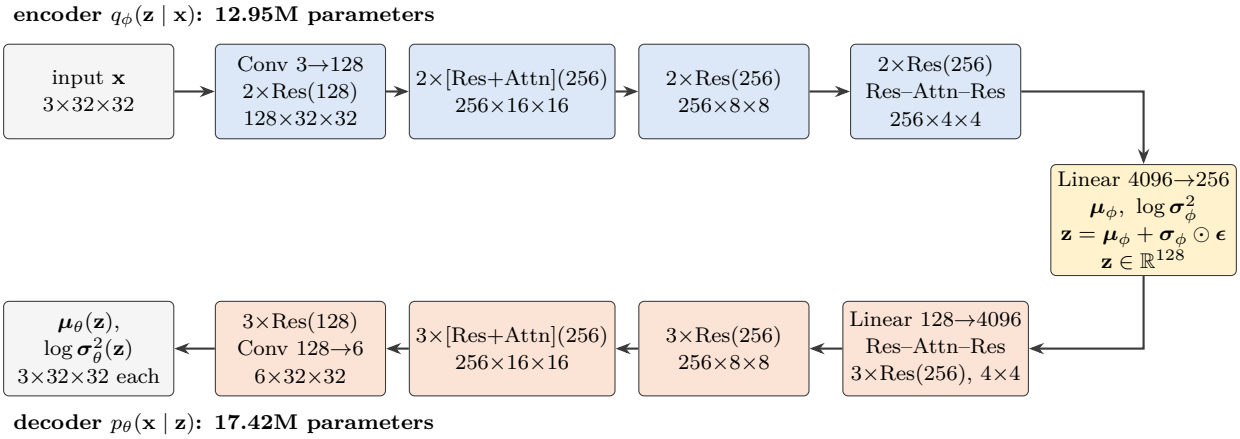


Figure 3: VAE-Conv. The encoder (top) mirrors the U-Net's downsampling path and the decoder (bottom) its upsampling path, using the same blocks and widths but *without skip connections*: all information must pass through the 128-d latent $\mathbf{z}$ (right).

A one-layer MLP has 4.9M parameters and no spatial inductive bias. Compared with a 35.7M-parameter U-Net, it would turn the comparison into "MLP vs. U-Net". VAE-Conv therefore keeps the VAE *method* of Section 2.2 exactly and replaces only the *network*, with an encoder and decoder built from the blocks of Section 3.1 at the U-Net's widths (base width 128, channel multipliers $(1, 2, 2, 2)$, two residual blocks per level, attention at 16×16).

Encoder, step by step.

1. A 3×3 convolution lifts the image to 128 channels.
2. Four resolution levels ($32 \to 16 \to 8 \to 4$) apply two residual blocks each; the 16×16 level adds self-attention after each block.
3. Stride-2 convolutions downsample between levels.
4. A bottleneck (residual block, attention, residual block) operates at 4×4.
5. GroupNorm and SiLU are applied, and the result is flattened to $256 \cdot 4 \cdot 4 = 4096$ values.
6. A linear layer outputs $\boldsymbol{\mu}_\phi(\mathbf{x})$ and $\log \boldsymbol{\sigma}^2_\phi(\mathbf{x})$ (2×128 values). The latent is sampled with the reparameterisation trick.

Decoder, step by step.

1. A linear layer maps $\mathbf{z} \in \mathbb{R}^{128}$ to a $256 \times 4 \times 4$ tensor.
2. The same bottleneck is applied.
3. Four levels with *three* residual blocks each (mirroring the U-Net's upsampling path) upsample back to 32×32.
4. GroupNorm, SiLU and a 3×3 convolution produce 6 channels: the per-pixel mean $\boldsymbol{\mu}_\theta(\mathbf{z})$ and raw log-variance $\mathbf{r}(\mathbf{z})$ of the Gaussian decoder.

Decoder variance. With dequantised data, a log-variance far below the variance of the dequantisation noise brings no real gain in likelihood and is numerically unstable. We therefore use a soft

Table 2: VAE-Conv layers (output shapes are channels × height × width). Encoder 12,950,400 + decoder 17,421,190 = **30,371,590** parameters.

Part	Stage	Output	Parameters
Encoder	Conv 3×3, $3 \to 128$	$128 \times 32 \times 32$	3,584
	Level 0: 2×Res(128) + downsample	$128 \times 16 \times 16$	738,944
	Level 1: 2×[Res + Attn](256) + downsample	$256 \times 8 \times 8$	3,217,664
	Level 2: 2×Res(256) + downsample	$256 \times 4 \times 4$	2,952,448
	Level 3: 2×Res(256)	$256 \times 4 \times 4$	2,362,368
	Bottleneck: Res – Attn – Res	$256 \times 4 \times 4$	2,626,048
	GN, SiLU, flatten	4096	512
	Linear $4096 \to 2 \times 128$ ($\boldsymbol{\mu}_\phi$, $\log \boldsymbol{\sigma}^2_\phi$)	2×128	1,048,832
Decoder	Linear $128 \to 4096$, reshape	$256 \times 4 \times 4$	528,384
	Bottleneck: Res – Attn – Res	$256 \times 4 \times 4$	2,626,048
	Level 3: 3×Res(256) + upsample	$256 \times 8 \times 8$	4,133,632
	Level 2: 3×Res(256) + upsample	$256 \times 16 \times 16$	4,133,632
	Level 1: 3×[Res + Attn](256) + upsample	$256 \times 32 \times 32$	4,924,672
	Level 0: 3×Res(128)	$128 \times 32 \times 32$	1,067,648
	GN, SiLU, Conv 3×3, $128 \to 6$ ($\boldsymbol{\mu}_\theta$, $\mathbf{r}$)	$6 \times 32 \times 32$	7,174

lower bound,

$$\log \boldsymbol{\sigma}^2_\theta(\mathbf{z}) = \ell_{\min} + \operatorname{softplus}\big(\mathbf{r}(\mathbf{z}) - \ell_{\min}\big), \qquad \ell_{\min} = -12, \tag{27}$$

which lies at the level of the dequantisation noise. The same bound is used in VAE-MLP.

3.4 Model C — DDPM: the noise-prediction U-Net of Ho et al.

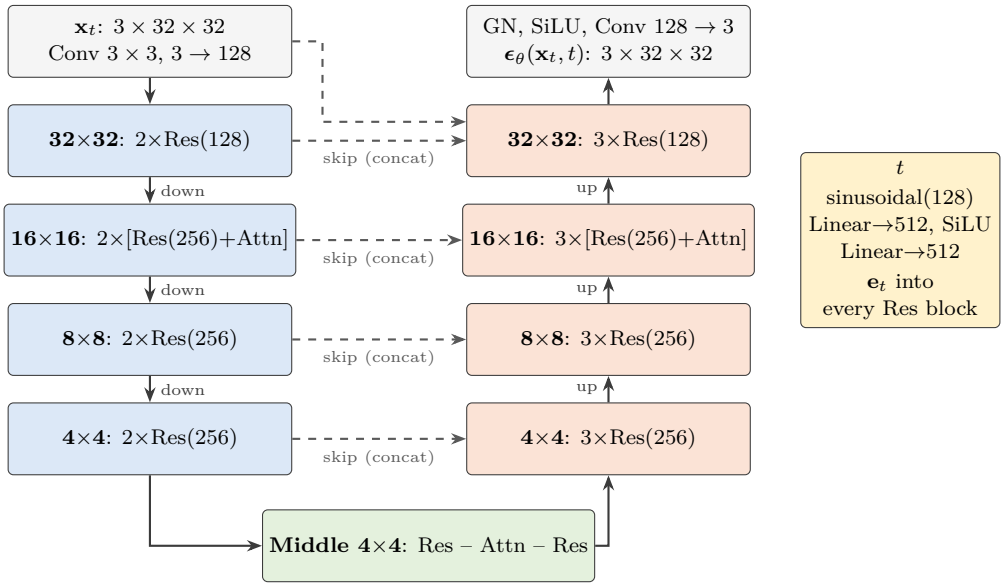


Figure 4: The DDPM U-Net [10, 11]. The left path stores 12 activations (the input convolution, every residual block and every downsampling). The right path concatenates one of them before each of its residual blocks (dashed arrows; one is drawn per level). The timestep embedding $\mathbf{e}_t$ conditions every residual block. Total: 35,746,307 parameters.

The DDPM network $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ maps a noisy image and its timestep to a prediction of the added noise, so input and output have the same shape. We reproduce the CIFAR-10 configuration of Ho et al. [2] (Appendix B): a PixelCNN++-style U-Net on a Wide-ResNet backbone with GroupNorm, four resolutions, two residual blocks per level on the way down, self-attention at 16×16, dropout

Table 3: DDPM U-Net (CIFAR-10 configuration). Residual blocks on the expanding path receive concatenated skip features, so their input width is shown as $C_{\text{in}} \to C_{\text{out}}$. Total: **35,746,307** parameters (paper: 35.7M).

Path	Stage	Output	Parameters
	Timestep MLP: sinusoidal(128) $\to$ 512 $\to$ 512	512	328,704
	Conv 3×3, $3 \to 128$	$128 \times 32 \times 32$	3,584
Down	32×32: 2×Res $128 \to 128$ + downsample	$128 \times 16 \times 16$	870,272
	16×16: 2×[Res + Attn], $128 \to 256$ + downsample	$256 \times 8 \times 8$	3,480,320
	8×8: 2×Res $256 \to 256$ + downsample	$256 \times 4 \times 4$	3,215,104
	4×4: 2×Res $256 \to 256$	$256 \times 4 \times 4$	2,625,024
Middle	Res – Attn – Res (256)	$256 \times 4 \times 4$	2,888,704
Up	4×4: 3×Res $512 \to 256$ + upsample	$256 \times 8 \times 8$	6,692,608
	8×8: 3×Res $512 \to 256$ + upsample	$256 \times 16 \times 16$	6,692,608
	16×16: 3×[Res + Attn] $(512, 512, 384 \to 256)$ + upsample	$256 \times 32 \times 32$	7,155,712
	32×32: 3×Res $(384, 256, 256 \to 128)$	$128 \times 32 \times 32$	1,789,952
	GN, SiLU, Conv 3×3, $128 \to 3$	$3 \times 32 \times 32$	3,715

0.1, and the sinusoidal timestep embedding added to every residual block.

How the U-Net processes an input, step by step.

1. **Time conditioning.** t is embedded into $\mathbf{e}_t \in \mathbb{R}^{512}$, which every residual block adds after its first convolution (Equation (20)). One set of weights can therefore denoise at all 1000 noise levels.
2. **Contracting path.** A 3×3 convolution lifts $\mathbf{x}_t$ to 128 channels. Four levels $(32, 16, 8, 4)$ each apply two residual blocks, with attention at 16×16, followed by stride-2 downsampling. Every intermediate activation is stored for later.
3. **Middle.** A residual block, global self-attention and another residual block at 4×4 give every position a view of the whole image.
4. **Expanding path.** At each level, three residual blocks each take the current features *concatenated* with one stored activation of the same resolution. This is why their input widths (e.g. $512 = 256 + 256$ or $384 = 256 + 128$) exceed their outputs. Nearest-neighbour upsampling with a convolution doubles the resolution between levels.
5. **Output.** GroupNorm, SiLU and a zero-initialised 3×3 convolution produce $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$.

The skip connections are the key structural difference from VAE-Conv. They let fine detail bypass the bottleneck, which is appropriate here because the DDPM never needs to compress: its "latent" $\mathbf{x}_t$ is as large as the image.

3.5 The three architectures side by side

Table 4: Structural comparison of the three models.

	VAE-MLP	**VAE-Conv**	**DDPM**
Network	2 hidden layers (1 per side), tanh	24 residual + 7 attention blocks, GN + SiLU	U-Net: 22 residual + 6 attention blocks
Parameters	4.9M	30.4M	35.7M
Spatial inductive bias	none (flattened pixels)	convolutions + attention	convolutions + attention
Encoder	learned, amortised	learned, amortised	fixed (Gaussian noising), no parameters
Latent	$\mathbf{z} \in \mathbb{R}^{200}$ (compressed 15×)	$\mathbf{z} \in \mathbb{R}^{128}$ (compressed 24×)	$\mathbf{x}_1, \ldots, \mathbf{x}_{1000}$, each $\in \mathbb{R}^{3072}$
Skip connections	no	no (bottleneck)	yes (12 U-Net skips)
Conditioning	none	none	timestep t in every block
Network evaluations per sample	1	1	1000 (10–250 with DDIM)
Training objective	ELBO, $\beta = 1$	ELBO, $\beta = 1$	L_{simple} (reweighted ELBO)

VAE-MLP and VAE-Conv share every component of the method and differ *only* in the network, so comparing them isolates the effect of architecture. VAE-Conv and the DDPM share building blocks, widths and training settings and differ in the *generative approach*, so comparing them isolates the effect of VAE versus diffusion.

4 Training Procedure

4.1 Algorithms

Algorithms 1 to 4 summarise how each model is trained and sampled. All three models use the same training loop (`src/genmodels/trainer.py`), with bf16 mixed precision, gradient clipping, an exponential moving average (EMA) of the weights, and checkpoints that can be resumed exactly.

Algorithm 1 VAE training (one step, both VAE variants)

1: Sample a minibatch of pixel values $\{k^{(b)}\}_{b=1}^{B}$
2: Dequantise: $\mathbf{x}^{(b)} \leftarrow (k^{(b)} + \mathbf{u})/128 - 1, \;\; \mathbf{u} \sim \mathcal{U}[0,1)^D$
3: Encode: $\boldsymbol{\mu}_\phi, \log \boldsymbol{\sigma}_\phi^2 \leftarrow \text{Enc}_\phi(\mathbf{x}^{(b)})$
4: Reparameterise: $\mathbf{z} \leftarrow \boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon}, \;\; \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Decode: $\boldsymbol{\mu}_\theta, \log \boldsymbol{\sigma}_\theta^2 \leftarrow \text{Dec}_\theta(\mathbf{z})$
6: $\mathcal{B}^{(b)} \leftarrow$ reconstruction term (6) + KL term (7) ▷ both summed over all dimensions
7: Gradient step on $\frac{1}{BD}\sum_b \mathcal{B}^{(b)}$; update the EMA weights

Algorithm 2 DDPM training with L_{simple} [2]

Sample a minibatch $\mathbf{x}_0$ (scaled to $[-1, 1]$, random horizontal flips)
$t \sim \mathcal{U}\{1, \dots, T\}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ▷ independently for each image
$\mathbf{x}_t \leftarrow \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$ ▷ Equation (11): one shot, no chain
Gradient step on $\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right\|^2$; update the EMA weights

Algorithm 3 DDPM ancestral sampling (paper's sampler, $T = 1000$ network evaluations)

$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $t = T, \dots, 1$ **do**
    $\hat{\mathbf{x}}_0 \leftarrow \text{clip}_{[-1,1]}\big((\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t))/\sqrt{\bar{\alpha}_t}\big)$
    $\mathbf{x}_{t-1} \leftarrow \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \hat{\mathbf{x}}_0) + \sigma_t\,\mathbf{z}$, $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$ ▷ Equation (13)
return $\mathbf{x}_0$

Algorithm 4 DDIM sampling [5] ($\eta = 0$, S steps, same trained network)

$\tau \leftarrow (0, c, 2c, \dots)$ with $c = T/S$; $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $i = S, \dots, 1$ **do**
    $\hat{\boldsymbol{\epsilon}} \leftarrow \boldsymbol{\epsilon}_\theta(\mathbf{x}, \tau_i)$, $\hat{\mathbf{x}}_0 \leftarrow \text{clip}\left((\mathbf{x} - \sqrt{1-\bar{\alpha}_{\tau_i}}\hat{\boldsymbol{\epsilon}})/\sqrt{\bar{\alpha}_{\tau_i}}\right)$
    $\hat{\boldsymbol{\epsilon}} \leftarrow (\mathbf{x} - \sqrt{\bar{\alpha}_{\tau_i}}\,\hat{\mathbf{x}}_0)/\sqrt{1-\bar{\alpha}_{\tau_i}}$ ▷ keep $\hat{\boldsymbol{\epsilon}}$ consistent with the clipped $\hat{\mathbf{x}}_0$
    $\mathbf{x} \leftarrow \sqrt{\bar{\alpha}_{\tau_{i-1}}}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_{\tau_{i-1}}}\,\hat{\boldsymbol{\epsilon}}$ ▷ Equation (18); $\bar{\alpha}_{\tau_0} := 1$
return $\mathbf{x}$

4.2 Hyper-parameters and compute

Table 5: Training configuration (`configs/*.yaml`). The DDPM settings are those of Ho et al. [2] except the number of steps. VAE-Conv uses the DDPM's optimisation settings, for fairness. VAE-MLP uses the settings of Kingma and Welling [1].

	VAE-MLP	**VAE-Conv**	**DDPM**
Optimiser	Adagrad, lr 0.02	Adam, lr 2×10^{-4}	Adam, lr 2×10^{-4}
Warm-up / gradient clipping	– / –	5k steps / 1.0	5k steps / 1.0
Batch size	100	128	128
Training steps (epochs)	500k (1000)	60k (154)	150k (384); paper: 800k
Regularisation	weight decay from $p(\theta) = \mathcal{N}(\mathbf{0}, \mathbf{I})$	–	dropout 0.1
Augmentation	none	horizontal flips	horizontal flips
EMA decay	0.9999	0.9999	0.9999
Initialisation	$\mathcal{N}(0, 0.01^2)$	Xavier (fan-avg)	Xavier (fan-avg)
Precision	fp32	bf16 autocast	bf16 autocast
Training time (A100-80GB)	0.44 h	1.32 h	3.94 h

All models were trained on a single NVIDIA A100-SXM4-80GB (Google Colab). The total training compute was 5.72 GPU-hours, including a 0.02-hour CIFAR-10 classifier used as an evaluation instrument.

The DDPM is the only model whose training length deviates from its paper: 150k instead of 800k steps, which the compute budget required. For reference, Ho et al. [2] trained for 10.6 hours on a TPU v3-8, roughly comparable to 8 V100 GPUs. Section 9 discusses the effect on the results.

5 Evaluation Protocol

Every model is evaluated by the same script (`scripts/evaluate.py`) on the same GPU.

- Each sample set has $N = 10{,}000$ images.
- The reference set is the 50,000 CIFAR-10 training images.
- FID, IS, KID and precision/recall all use the same Inception-v3 features: the TF-compatible network of `torch-fidelity` [12].

5.1 Metrics

Let $\{\mathbf{f}_i\}$ denote 2048-d Inception pool features of real (r) or generated (g) images.

- **Fréchet Inception Distance** [13]: the distance between Gaussians fitted to the two feature sets. It measures quality and diversity together.

$$\mathrm{FID} = \|\mathbf{m}_r - \mathbf{m}_g\|_2^2 + \mathrm{Tr}\Big(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2\big(\boldsymbol{\Sigma}_r\boldsymbol{\Sigma}_g\big)^{1/2}\Big). \tag{28}$$

- **Inception Score** [14]: high when each image is classified confidently and the classes are varied. It never compares against real CIFAR-10 images. 10 splits:

$$\mathrm{IS} = \exp\Big(\mathbb{E}_{\mathbf{x}}\, D_{\mathrm{KL}}(p(y \mid \mathbf{x}) \,\|\, p(y))\Big). \tag{29}$$

- **Kernel Inception Distance** [15]: an unbiased estimate of the squared MMD with kernel $k(\mathbf{a}, \mathbf{b}) = (\mathbf{a}^\top \mathbf{b}/2048 + 1)^3$, averaged over 100 subsets of 1000. Unlike FID, it is unbiased at our sample size.

- **Precision and recall** [16]: each real feature is assigned a radius equal to the distance to its k-th nearest real neighbour ($k = 3$), and likewise for generated features.

$$\text{precision} = \frac{1}{N_g}\sum_j \mathbb{1}\big[\mathbf{f}_j^g \in \textstyle\bigcup_i B(\mathbf{f}_i^r, r_i)\big], \qquad \text{recall} = \tfrac{1}{N_r}\textstyle\sum_i \mathbb{1}\big[\mathbf{f}_i^r \in \bigcup_j B(\mathbf{f}_j^g, r_j)\big]. \tag{30}$$

  Precision measures *realism*: generated images lie on the real data manifold. Recall measures *diversity*: generated images cover the real data. FID mixes the two.

- **Likelihood**: the test-set negative ELBO in bits/dim, using Equation (9) for the VAEs and the full bound of Equation (15) for the DDPM. Both are upper bounds on the negative log-likelihood of the discrete data, so they are directly comparable.

- **Cost**: network evaluations per image (NFE, hardware-independent), throughput in images/s at batch 256, and single-image latency, all in bf16 on the A100.

- **Thematic consistency**: a ResNet-18 trained from scratch on CIFAR-10 (93.6% test accuracy) labels every generated image. We report its mean confidence, the share of confident predictions (> 0.9), and the normalised entropy of the predicted-class histogram (1 = all classes equally frequent).

- **Memorisation**: for every sample, the Inception-feature distance to its nearest training image, relative to the same distance for unseen real test images.

5.2 Protocol and fairness

- **Validated metrics.** On real data, our pipeline gives FID($\text{test}_{10k}, \text{train}_{50k}$) = 3.15 and IS(test) = 10.97, matching published reference values. This was checked before any model was evaluated.
- **Real-data reference row.** Every metric is also reported for real test images, so the ceiling is visible.
- **Fixed checkpoints, no cherry-picking.** The final EMA checkpoint of each run was chosen before evaluation, and all sample grids are uncurated.
- **Evaluation noise.** Headline settings are evaluated on three independent sample sets and reported as mean ± standard deviation.
- **Same conditions.** All models use identical data, preprocessing, GPU and software.

6 Quantitative Results

6.1 Main comparison

Table 6: Main results (N = 10,000 samples against the CIFAR-10 training set). ±: standard deviation over three independent sample sets (evaluation noise). NFE: network evaluations per image. Bits/dim: test-set bound; it is a property of the model, so both DDPM rows share it.

Model	FID↓	IS↑	KID×10^3↓	Prec.↑	Rec.↑	Bits/dim↓	NFE	Img/s↑
Real test images	*3.15*	*10.97*	*−0.01*	*0.681*	*0.693*	–	–	–
VAE-MLP	135.97 ± 0.52	3.11 ± 0.02	127.57 ± 0.62	0.323 ± 0.006	0.003 ± 0.001	6.313	1	350,102
VAE-Conv	121.85 ± 0.30	3.88 ± 0.05	151.18 ± 0.45	0.404 ± 0.010	0.073 ± 0.020	5.826	1	7,017
DDPM, 1000 steps	**6.58**	**8.59**	**1.54**	**0.694**	0.583	**3.794**	1000	4.2
DDPM, DDIM 100 steps	9.66 ± 0.06	8.31 ± 0.10	6.35 ± 0.14	0.641 ± 0.007	**0.592** ± 0.008	3.794	100	41.6

- **The DDPM wins every quality metric by a wide margin.** Its FID is 18.5× lower than VAE-Conv's and its IS 2.2× higher. Its precision (0.694) equals that of real images (0.681), and its recall is 8× VAE-Conv's.
- **The VAE is about** 1,700× **faster** (7,017 vs. 4.2 images/s; 14 ms vs. 21.8 s for a single image).
- **Between the two VAEs,** the convolutional network is better on FID, IS, precision, recall and likelihood, but worse on KID. Both VAEs are far from the data distribution, where the metrics stop ranking models consistently, so we conclude only that VAE-Conv is better on most measures.

Comparison with the original paper.

- **Likelihood.** After 150k of the paper's 800k steps, our DDPM's test bound is 3.794 bits/dim with $\sigma_t^2 = \beta_t$, or 3.781 with $\sigma_t^2 = \tilde{\beta}_t$. The paper reports $\leq$ 3.75 for L_{simple} (2, Table 2). Agreement to within 0.04 bits/dim is strong evidence that the network, schedule and bound are implemented correctly.
- **FID and IS.** The paper reports 3.17 and 9.46 with 50k samples and its best checkpoint. Our 6.58 and 8.59 use 10k samples (which raises FID) and the final checkpoint. Training was also 5.3× shorter, and FID was still falling at the end of training (Figure 9).

6.2 Speed–quality trade-off

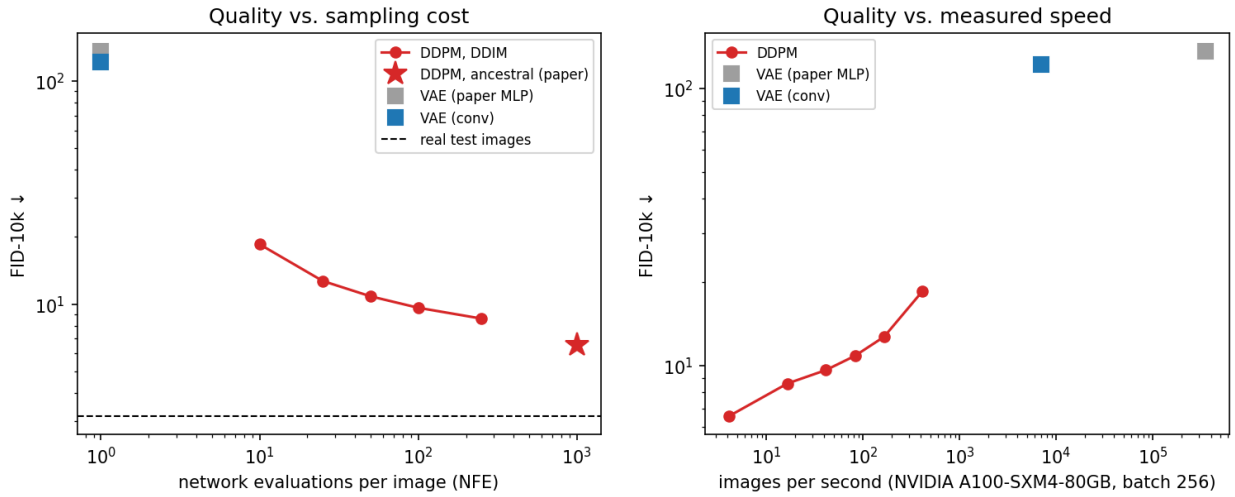


Figure 5: Left: FID against the number of network evaluations per image (log–log). Right: FID against measured throughput. DDIM traces a frontier with the *same* trained DDPM; the VAEs are single points at one evaluation.

Table 7: DDPM sampling steps (same network; DDIM sets share their starting noise).

Sampler	NFE	FID↓	IS↑	Precision	Recall	Img/s	Latency (1 image)
Ancestral (paper)	1000	**6.58**	**8.59**	**0.694**	0.583	4.2	21.75 s
DDIM	250	8.61	8.36	0.643	**0.595**	16.6	5.41 s
DDIM	100	9.63	8.24	0.639	0.584	41.6	2.20 s
DDIM	50	10.84	8.11	0.632	0.572	83.3	1.12 s
DDIM	25	12.70	8.02	0.626	0.555	166.5	0.55 s
DDIM	10	18.55	7.72	0.595	0.509	415.6	0.23 s
VAE-Conv	*1*	*121.85*	*3.88*	*0.404*	*0.073*	*7,017*	*14 ms*

- DDIM with 10 steps is 100× faster than the paper's sampler and still 6.6× better in FID than VAE-Conv.
- DDIM does not reach the ancestral sampler's quality even at 250 steps.
- No DDPM setting matches the VAE's speed: at 10 steps it is still 17× slower.

6.3 Quality versus diversity

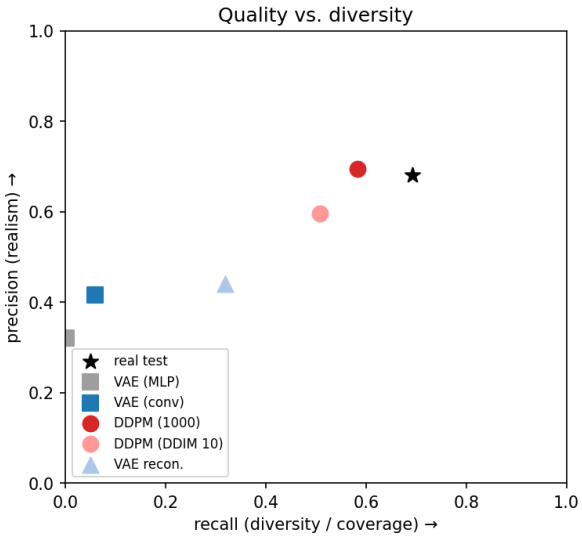


Figure 6: Precision (realism) against recall (diversity), both computed in Inception feature space. Real test images (star) mark the ceiling. The DDPM sits next to them; both VAEs are in the low-recall corner. VAE-Conv's reconstructions (triangle) cover far more of the data than its samples do.

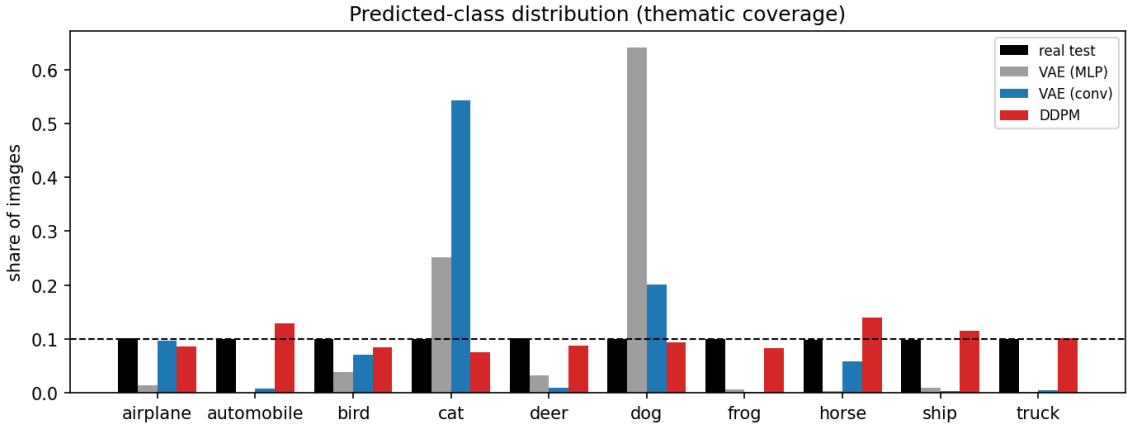


Figure 7: Distribution of the classes predicted by the CIFAR-10 classifier for each model's samples. Dashed line: the uniform share of 0.1 that real data has.

The DDPM lies next to the real-data point in Figure 6, while both VAEs sit in the low-recall corner. The VAEs' main failure is therefore *coverage*, not only blur. VAE-Conv's *reconstructions* reach a recall of 0.32, which points to the cause (Section 6.4).

Table 8: Thematic consistency, measured by a CIFAR-10 classifier (93.6% accuracy on real test images).

Images	Mean conf.↑	Conf. > 0.9 ↑	Class entropy (1 = even)	Rarest class share
Real test images	0.875	0.742	1.000	0.098
VAE-MLP	0.756	0.298	0.455	0.000
VAE-Conv	0.692	0.164	0.601	0.003
DDPM (1000 steps)	**0.865**	**0.719**	**0.991**	**0.076**
DDPM (DDIM 100)	0.854	0.670	0.995	0.080

- **DDPM samples are almost as recognisable as real photographs** (confidence 0.865 vs. 0.875), and all ten classes appear in nearly equal shares.
- **The VAEs' class distribution collapses.** The classifier labels 54% of VAE-Conv samples

"cat" and 64% of VAE-MLP samples "dog", while automobile, frog, ship and truck almost vanish from VAE-Conv's samples.

- **This does not mean the VAE "draws cats".** Its blurry, brownish samples fall *between* classes, and an uncertain classifier assigns them to broad animal classes. Only 16% of VAE-Conv samples are classified with confidence above 0.9, against 72% for the DDPM.

6.4 Training dynamics and the VAE's prior hole

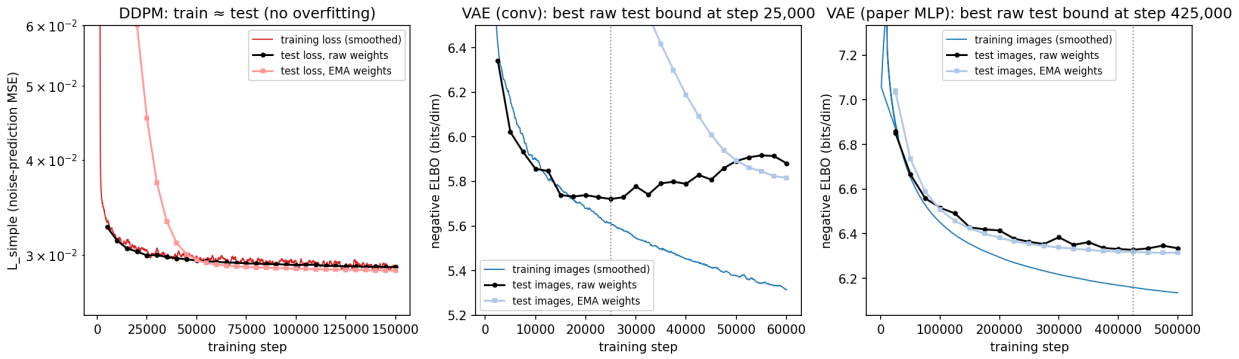


Figure 8: Training and test objectives. DDPM: training and test losses coincide, so there is no overfitting. VAE-Conv: the test bound is best at step 25k and then rises while the training bound keeps falling. VAE-MLP overfits only slightly.

- **VAE-Conv overfits its likelihood.** Its best test bound was 5.72 bits/dim at step 25k. By step 60k the training bound had fallen to 5.32 while the raw-weight test bound had risen to 5.88; the EMA weights we evaluate reach 5.82.
- **VAE-MLP overfits only slightly:** 6.14 bits/dim on training images vs. 6.32 on test images at the end of training.
- **The DDPM does not overfit.** Its final test loss (0.0289) matches its training loss (0.0292). Its EMA weights start far worse than the raw weights because, as in the paper, the EMA (decay 0.9999) has no warm-up and still contains much of the random initialisation early on. They catch up after about 45k steps.

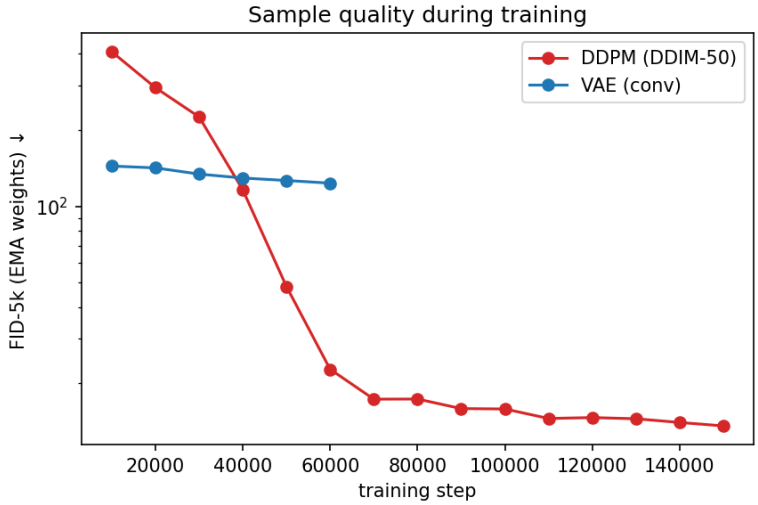


Figure 9: FID during training (5k samples, EMA weights; DDPM sampled with DDIM-50). The DDPM's FID keeps improving long after its loss has flattened, and was still falling at 150k steps.

Likelihood and sample quality disagree.

- VAE-Conv's FID kept improving (144 → 123) while its test likelihood got worse.
- The DDPM's loss was flat from about 20k steps, while its FID fell from 408 (10k steps) to 22.7 (60k) and 13.6 (150k).

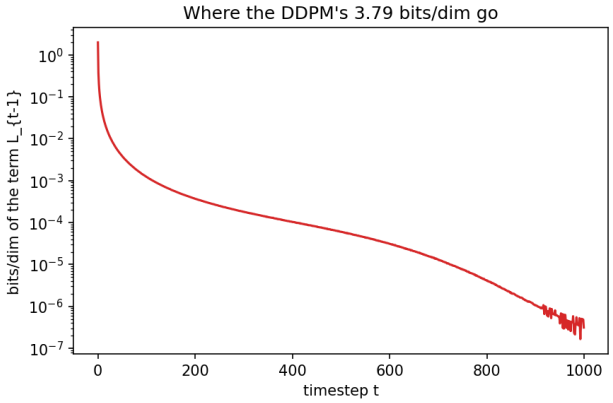
This illustrates the observation of Theis et al. [17] that likelihood and sample quality are distinct objectives.

The prior hole, measured.

- VAE-Conv's *reconstructions* of test images reach FID 57.9 and recall 0.32.
- Its *samples* from $p(\mathbf{z})$ reach FID 121.9 and recall 0.07.

The decoder can produce far better images than it does when sampling from the prior. The reason is that the aggregate posterior $q_\phi(\mathbf{z}) = \mathbb{E}_{\mathbf{x}}\, q_\phi(\mathbf{z} \mid \mathbf{x})$ does not fill $\mathcal{N}(\mathbf{0}, \mathbf{I})$, so a random $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ often lands where the decoder never received training signal. This is not posterior collapse: all 128 latent dimensions are active [18], with a KL of 381 nats. The DDPM avoids the problem by construction: its prior term L_T is 8×10^{-6} bits/dim.

6.5 Likelihood decomposition



Model	Test bits/dim↓
VAE-MLP (ELBO)	6.313 ± 0.006
VAE-Conv (ELBO)	5.826 ± 0.007
DDPM, $\sigma_t^2 = \beta_t$	3.794 ± 0.015
DDPM, $\sigma_t^2 = \tilde{\beta}_t$	3.781 ± 0.019

±: standard error. DDPM: 1,000 test images (1000 network evaluations each); VAEs: all 10,000.

Figure 10: Left: contribution of each term L_{t-1} of Equation (15) to the DDPM's 3.79 bits/dim. Right: test likelihood bounds of all models.

Of the DDPM's 3.79 bits/dim:

- 1.99 (52%) are spent on the final decoder term L_0;
- 3.57 (94%) are spent on the last 50 of the 1000 steps;
- the first ~ 900 steps, where the content of the image is decided (Section 7), cost almost nothing.

Most of the code length therefore describes imperceptible detail, which explains how a model can have excellent samples without a state-of-the-art likelihood. This matches the interpretation in Section 4.3 of Ho et al. [2]. For comparison, VAE-Conv spends 0.18 bits/dim (381 nats per image) on the KL term that keeps its posterior close to the prior, while the DDPM's prior term L_T is negligible.

6.6 Ablations without extra training, and memorisation

Table 9: Left: effect of EMA weights (DDIM-100 for the DDPM). Right: memorisation check, i.e. nearest-neighbour distance to the training set relative to unseen real test images, and the share of samples closer than the real test images' 1st percentile (a copying model would score $\gg 0.01$).

	FID↓	IS	Prec.	Rec.
DDPM, EMA	**9.63**	**8.24**	**0.639**	0.584
DDPM, raw	19.50	7.53	0.568	**0.608**
VAE-Conv, EMA	121.51	3.87	0.415	0.059
VAE-Conv, raw	**118.90**	**3.90**	0.412	**0.076**

	NN ratio	$<$ 1st pct.
VAE-MLP	1.302	0.0000
VAE-Conv	1.337	0.0000
DDPM	1.024	0.0009

- **EMA halves the DDPM's FID.** For VAE-Conv it slightly *hurts* FID, by more than the evaluation noise, although it gives the better test likelihood. This is one more instance of likelihood and sample quality disagreeing.
- **No memorisation.** DDPM samples are about as close to the training set as unseen real images are, and only 0.09% are unusually close. Figure 11 shows the DDPM's eight closest samples: they share pose and subject with their nearest training images but differ in colour, background and detail. The VAE's samples are far from every training image because they are blurry.

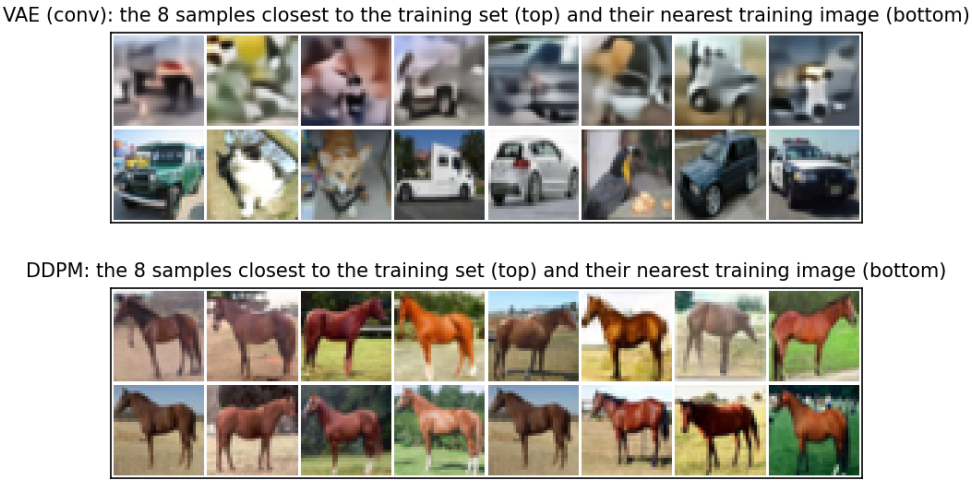


Figure 11: The eight samples closest to the training set (top row of each pair) and their nearest training images (bottom row), in Inception feature space. Even the closest DDPM samples are new images, not copies.

7 Qualitative Analysis

All grids are **uncurated**: they show the first samples of a fixed random seed.



Figure 12: Uncurated samples (first 100 of a fixed seed) of each model next to real CIFAR-10 test images.

7.1 Realism

- **DDPM.** Most samples are sharp, coherent photographs with recognisable objects (horses, cars, trucks, ships, birds, frogs), plausible lighting and backgrounds, and correct global structure. Failures occur at the level of objects (implausible shapes, objects blending into the background, ambiguous species), while low-level texture is almost always realistic.
- **VAE-Conv.** Samples have plausible colours and rough layouts, typically a foreground blob on a background, but no sharp edges or texture.
- **VAE-MLP.** Samples are blurrier still: high-contrast colour blobs with little object structure.

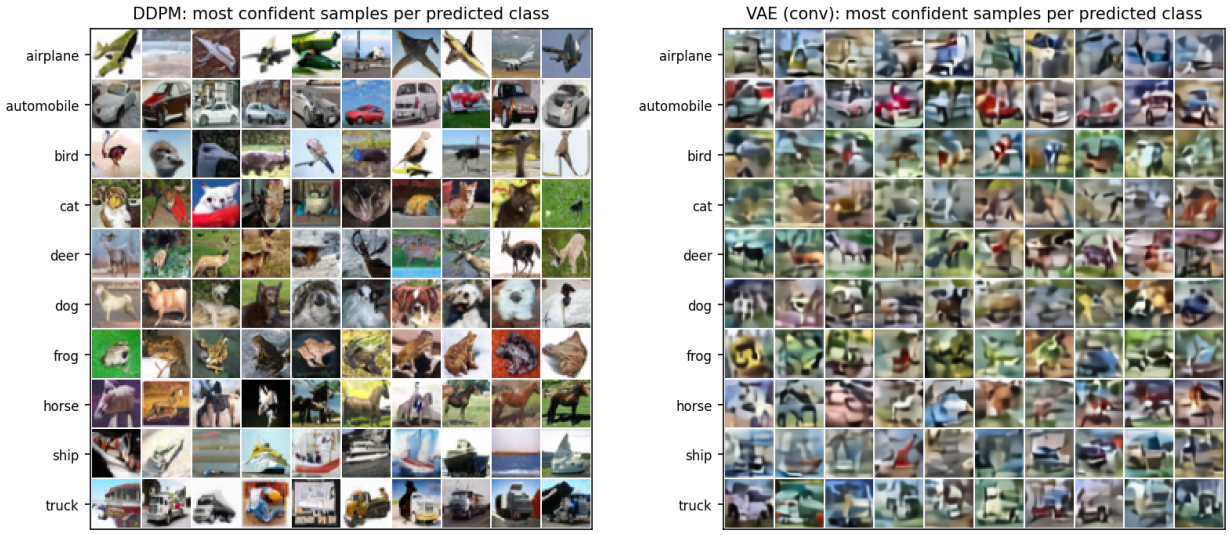
The mechanism follows from Equation (6). The decoder models pixels as independent Gaussians, so when many sharp images are compatible with one latent, the optimal mean is their *average*, which is blurry. Each reverse step of the DDPM, in contrast, only removes a small amount of noise. That is a nearly Gaussian problem which a Gaussian transition models accurately, so detail survives.



Figure 13: The 32 least recognisable samples according to the classifier. The VAE's worst samples are abstract and blurry. The DDPM's worst samples are still sharp, photograph-like images whose *content* is ambiguous, as in many real CIFAR-10 images.

7.2 Diversity and thematic consistency

The DDPM grid varies in class, colour, background, scale and viewpoint about as much as the real grid does (recall 0.58 vs. 0.69). The VAE grids repeat similar muted colours and centred blobs (recall 0.07).



(a) DDPM **(b)** VAE-Conv

Figure 14: The classifier's most confident samples for each predicted class. DDPM samples are clear instances of their class; even the VAE's most confident samples are only loosely related to theirs.

7.3 How each model generates

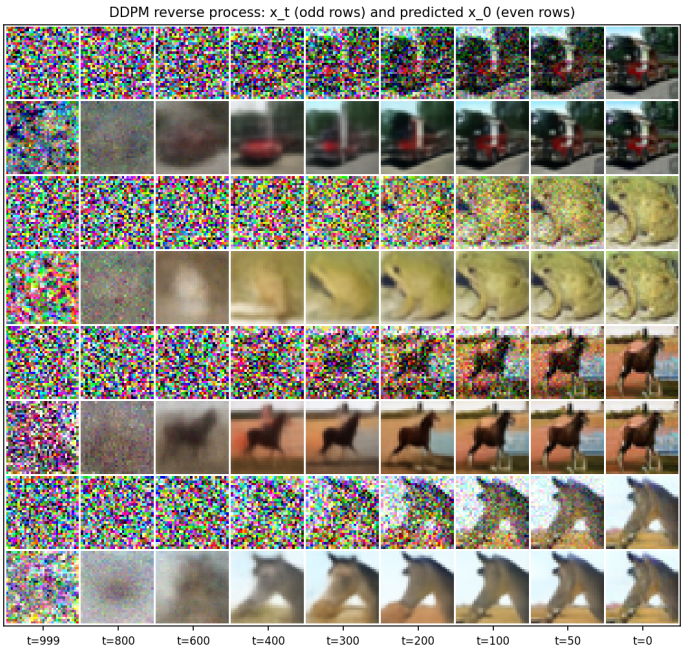


Figure 15: The DDPM reverse process. Odd rows show $\mathbf{x}_t$ and even rows the predicted clean image $\hat{\mathbf{x}}_0$. Generation is *coarse-to-fine*: layout and colours are fixed by $t \approx 600$–400 (a red truck, a frog, a horse), and later steps only add detail. Together with Figure 10, this shows that the content is decided early and cheaply, while most of the likelihood is spent on fine detail.



Figure 16: VAE reconstructions. Rows: test images, VAE-Conv, VAE-MLP. Layout and colour survive the 128-d bottleneck; texture does not.

The two VAEs reach almost the same PSNR (24.07 vs. 23.61 dB) and SSIM (0.772 vs. 0.771) on test reconstructions. Yet VAE-MLP's reconstructions are visibly blurrier, and their FID is far worse (99.4 vs. 57.9). Pixel-wise metrics do not capture perceptual quality.

7.4 Latent spaces



Figure 17: Interpolations. Rows 1–3: linear interpolation between the VAE latents of two test images gives smooth, gradual blends; the VAE has a learned, semantic latent space. Rows 4–6: spherical interpolation between two DDPM starting noises $\mathbf{x}_T$, decoded deterministically with DDIM-50, gives sharp images throughout. The content, however, *switches* between objects rather than blending, because the DDPM's "latent" is simply noise.

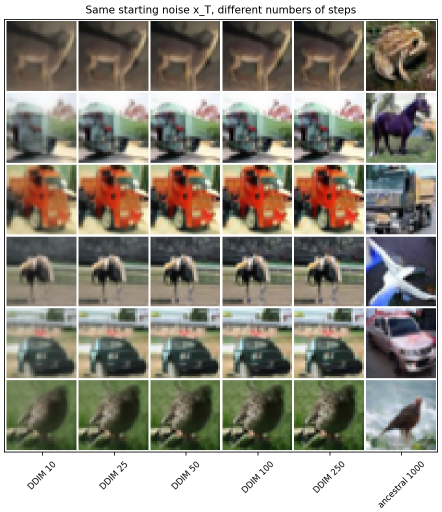


Figure 18: The same starting noise $\mathbf{x}_T$ decoded with 10–250 DDIM steps, and with the 1000-step ancestral sampler (last column). DDIM is deterministic, so every step count gives the *same* image and extra steps only sharpen details. The ancestral sampler adds fresh noise at every step, so its image differs. This is expected, not a bug.

7.5 Blind rating by a vision-language model

As an independent, subjective judgement, we ran a blind rating of 150 shuffled images: 50 real CIFAR-10 test images, 50 VAE-Conv samples and 50 DDPM samples (1000-step sampler), drawn at random from the evaluated sample sets. The rater was **Gemini**, a vision-language model, not a person. For every image it gave:

- a realism score from 1 to 5;
- the CIFAR-10 class, or "unrecognisable";
- a verdict of real or generated.

The instructions named neither the models nor the proportions of each source, and stated that low resolution alone is no sign of being generated.

Table 10: Blind rating by Gemini (50 images per source; 95% Wilson intervals in brackets).

Source	Mean realism (1–5)	Named a class	Judged "real"	Correct class
Real test images	**4.58**	100%	96% [87, 99]	96%
DDPM	4.28	98%	**96% [87, 99]**	–
VAE-Conv	1.34	24%	0% [0, 7]	–

- **Gemini could not tell DDPM samples from real photographs.** It judged 48 of 50 in each group to be real, so its real-vs-DDPM accuracy is 50%, i.e. chance.
- **Its graded scores still detect a small gap.** Realism averaged 4.28 for DDPM vs. 4.58 for real photos (Mann–Whitney $p = 0.015$), consistent with the remaining FID gap. DDPM samples also received fewer top scores of 5 (20 vs. 33 of 50).
- **Every VAE sample was identified as generated.** 33 of the 50 VAE samples received the lowest realism score, and only 24% could be assigned any class, independently confirming Table 8.
- **Almost all DDPM samples were recognisable** (98% were assigned a class), and Gemini's labels cover all ten classes.

Integrity. All 50 "real" images are exact CIFAR-10 test images with the labels in the answer key, and none of the 100 generated images appears in the test set. The rater's answers contradict the key exactly where a blind rater would err: it labelled 48 of 50 DDPM samples "real". We report this single automated rater as supporting evidence, not as a substitute for a human study.

8 Discussion: the Main Differences between the Three Models

8.1 VAE-MLP vs. VAE-Conv: the effect of the network

The two VAEs share the method and differ only in the network (Table 4).

What the convolutional network improves.

- **Likelihood:** 5.83 vs. 6.31 bits/dim, i.e. a 0.49 bits/dim better bound.
- **Sample quality:** FID 121.9 vs. 136.0, and IS 3.88 vs. 3.11.
- **Coverage:** recall 0.073 vs. 0.003.
- **Reconstructions:** FID 57.9 vs. 99.4.

Convolutions share filters across positions, and attention relates distant positions. Both give the model the spatial inductive bias that the MLP, which treats an image as 3072 unrelated numbers, lacks.

What it does not change. Both VAEs remain in the same regime: blurry, low-recall samples, a large gap between reconstructions and samples, and a collapsed class distribution. **A better network improves a VAE but does not remove its characteristic failure modes.** The paper's own network is not the cause of the VAE's weaknesses on CIFAR-10. It only makes them worse.

8.2 VAE-Conv vs. DDPM: the effect of the generative approach

These two models share building blocks, widths, data and optimisation settings (Tables 4 and 5), and their sizes are comparable (30.4M vs. 35.7M parameters). Their difference is therefore attributable to the approach. Following Section 2.4, three structural differences explain all the measurements:

1. **One factorised step vs. many small steps.** The VAE must model the whole image distribution with a single Gaussian with independent pixels, which forces blurry averages. That gives low precision, low IS, and a realism score of 1.34 in the blind rating. Each DDPM step only has to model a small denoising update, for which a Gaussian is accurate. That gives precision equal to real data, and samples a vision-language model cannot tell from real photographs.
2. **Learned compressive encoder vs. fixed non-compressive encoder.** The VAE's encoder must squeeze the image into 128 numbers and match $\mathcal{N}(\mathbf{0}, \mathbf{I})$ while doing so. It never matches the prior perfectly, and that prior hole shows as samples far worse than reconstructions (FID 121.9 vs. 57.9), collapsed recall, and samples that fall between classes. The DDPM's forward process matches its prior by construction ($L_T \approx 8 \times 10^{-6}$ bits/dim) and compresses nothing.
3. **Cost of generation.** Both advantages of the DDPM are paid for at sampling time: 1000 network evaluations instead of 1, so about 1,700× slower. DDIM cuts this by up to 100× with no retraining, but even 10 steps remain 17× slower than the VAE.

8.3 Summary

Table 11: The three models at a glance (best per row in bold).

	VAE-MLP	**VAE-Conv**	**DDPM**
Sample quality (FID / IS)	136.0 / 3.11	121.9 / 3.88	**6.58 / 8.59**
Realism / diversity (prec. / rec.)	0.32 / 0.003	0.40 / 0.07	**0.69 / 0.58**
Likelihood (bits/dim)	6.31	5.83	**3.79**
Sampling speed (img/s)	**350,102**	7,017	4.2 (DDIM-10: 416)
Training time (GPU-h)	**0.44**	1.32	3.94
Learned encoder / latent code	yes, 200-d	yes, 128-d	no (latent = noise)
Typical failure	blurry colour blobs	blurry, between-class blobs	odd object shapes, sharp texture
Best use	teaching baseline	fast sampling, representation learning, compression	high-fidelity generation

Which model to use? It depends on the requirement:

- **Sample quality, diversity, class coverage or likelihood:** the DDPM is decisively better.
- **Real-time generation, or an encoder for compression or editing:** only the VAE provides these.

Modern latent diffusion models [19] combine the two: diffusion runs inside the compact latent space of an autoencoder. Our measurements show why this pairing works. The autoencoder contributes the cheap, compressed representation the DDPM lacks, and diffusion supplies the sample quality and prior matching the VAE lacks.

Lessons about evaluation.

- Likelihood and sample quality moved in opposite directions three times: VAE overfitting, the EMA ablation, and the DDPM's flat loss while its FID kept falling.
- IS alone would hide the VAE's diversity failure, and FID alone would hide the split between realism and diversity; precision/recall separates the two.
- Far from the data, the metrics stop ranking models consistently: FID and KID disagree on the two VAEs.

9 Limitations and Threats to Validity

Table 12 lists the remaining limitations of this study, how each could affect the conclusions, and how it is addressed. None of them changes the qualitative conclusions: the gap between the DDPM and the VAEs exceeds every source of uncertainty by more than an order of magnitude in FID.

Table 12: Remaining limitations, their effect, and how they are addressed.

Limitation	Effect on the conclusions	How it is addressed
Shortened DDPM training (150k of 800k steps)	Our DDPM numbers are a *lower bound* on what the implementation reaches; FID was still improving (Figure 9). It can only understate the DDPM's lead.	Disclosed as the only deviation from the paper (Table 5). The likelihood bound already matches the paper to within 0.04 bits/dim. Checkpoints are resumable, so training can be continued.
FID with 10k samples (papers: 50k)	FID is biased upward at smaller N, so our values are not directly comparable to published FID-50k.	The protocol is stated explicitly (Section 5) and applied identically to every model. KID, which is unbiased at this N, gives the same ranking between VAE and DDPM. Evaluation noise is measured on three independent sample sets.
Final checkpoint, not best	The paper reports its best FID over training, which is slightly optimistic.	Our choice was fixed in advance and avoids selection bias.
One training run per model	The $\pm$ values cover evaluation noise, not variance across training seeds. Small differences (e.g. between the two VAEs) should be read cautiously.	Conclusions rely only on differences far larger than the measured noise.
Classifier-based thematic metrics	They depend on one classifier, which behaves unpredictably on images far from real data (Table 8).	Interpreted cautiously, and cross-checked by the blind rating.
Automated blind rater	A single vision-language model is not a human study, and the answer key was accessible to it.	Reported as supporting evidence only. The answers contradict the key where a blind rater errs, and a human rating tool is included in the repository.
Original VAE method	Stronger VAE variants (hierarchical VAEs, learned priors) would narrow the gap.	Out of scope by design: the task asks for the original approaches.
Training not compiled	Our benchmark measured a 1.5× faster DDPM training step with `torch.compile` (16.1 vs. 10.6 steps/s), which was not enabled for the reported runs. This affects the training-time figures only, not the models or results.	Reported honestly as measured. Compilation is one configuration flag (`train.compile`) for future runs.
Code availability	A results report is only verifiable if the code is accessible.	All code, configurations, logs, raw evaluation outputs and a reproduction guide are public at https://github.com/Eyad-Elnakib1/Generative-models-VAE-DDPM.

10 Conclusion

We implemented a VAE and a DDPM from scratch, verified them against their papers, and compared three models on CIFAR-10 under one validated protocol.

1. **Faithful implementations.** The DDPM matches the published likelihood bound to within

0.04 bits/dim after 19% of the original training. Our metrics reproduce published reference values on real data.

2. **The DDPM wins decisively on quality:**
   - FID 6.58 vs. 121.9 and IS 8.59 vs. 3.88;
   - precision at the level of real data and 8× the recall;
   - even class coverage and 3.79 vs. 5.83 bits/dim;
   - no memorisation;
   - samples that a vision-language model could not distinguish from real photographs.
3. **The VAE wins decisively on speed and structure.** It needs one network evaluation per image (about 1,700× faster) and provides an encoder with a smooth, semantic latent space, which the DDPM lacks.
4. **The network matters, but does not change the regime.** Replacing the paper's MLP with a convolutional network improves every VAE metric, but leaves the VAE's blur, low diversity and prior hole intact.
5. **The main difference.** The VAE learns a compressed latent space and generates in one factorised step, which causes blur and a prior hole. The DDPM fixes its encoder and generates through many small, easy denoising steps, which gives quality at a high sampling cost. DDIM reduces that cost by up to 100× while still far outperforming the VAE.

References

[1] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014. arXiv:1312.6114.

[2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. arXiv:2006.11239.

[3] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning (ICML)*, 2015.

[4] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[5] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[6] Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision (ECCV)*, 2018.

[7] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. In *British Machine Vision Conference (BMVC)*, 2016.

[8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[11] Tim Salimans, Andrej Karpathy, Xi Chen, and Diederik P. Kingma. PixelCNN++: Improving the PixelCNN with discretized logistic mixture likelihood and other modifications. In *International Conference on Learning Representations (ICLR)*, 2017.

[12] Anton Obukhov, Maximilian Seitzer, Po-Wei Wu, Semen Zhydenko, Jonathan Kyl, and Elvis Yu-Jing Lin. High-fidelity performance metrics for generative models in PyTorch. https://github.com/toshas/torch-fidelity, 2020. Version 0.3.0.

[13] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[14] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training GANs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[15] Mikołaj Bińkowski, Danica J. Sutherland, Michael Arbel, and Arthur Gretton. Demystifying MMD GANs. In *International Conference on Learning Representations (ICLR)*, 2018.

[16] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved precision and recall metric for assessing generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[17] Lucas Theis, Aäron van den Oord, and Matthias Bethge. A note on the evaluation of generative models. In *International Conference on Learning Representations (ICLR)*, 2016.

[18] Yuri Burda, Roger Grosse, and Ruslan Salakhutdinov. Importance weighted autoencoders. In *International Conference on Learning Representations (ICLR)*, 2016.

[19] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

A Reproducibility

The full pipeline is available at https://github.com/Eyad-Elnakib1/Generative-models-VAE-DDPM. Every result in this report is produced by the following commands (on Google Colab, the notebook `notebooks/colab_runner.ipynb` runs the same steps):

```
pip install -e ".[eval]"
python scripts/train.py --config configs/vae_mlp_cifar10.yaml
python scripts/train.py --config configs/vae_conv_cifar10.yaml
python scripts/train.py --config configs/ddpm_cifar10.yaml
python scripts/train_classifier.py --out runs/classifier
```

```
python scripts/evaluate.py
python scripts/make_figures.py --model-figures
python scripts/plot_training_logs.py --runs runs \
       --out runs/eval/report/fig_training_curves.png
```

Verification. The repository contains 32 automated tests. They check the mathematics against independent derivations:

- the forward-process posterior against a numerical Bayes computation;
- exact recovery of the data by both samplers given a perfect noise predictor;
- normalisation of the discretised likelihood;
- the analytic KL term against a Monte-Carlo estimate;
- the ELBO's loss scaling ($\beta = 1$);
- bit-exact resumption of interrupted training runs;
- detection of mode collapse by precision/recall.

A further test checks that the Colab notebook contains exactly the repository's code. The executed notebook, including all training and evaluation logs, is part of the repository.

Table 13: Compute used (one NVIDIA A100-SXM4-80GB).

Run	GPU-hours
VAE-MLP, 500k steps	0.44
VAE-Conv, 60k steps	1.32
DDPM, 150k steps	3.94
Classifier (evaluation instrument), 30 epochs	0.02
Training total	**5.72**